

LAPORAN TUGAS
BAHASA PEMROGRAMAN 3
(Dosen : Dede Husen, M.Kom.)

Modul 8
Navigasi



Nama : Muhammad Rizal Nurfirdaus
NIM : 20230810088
Kelas : TINFC-2023-04

TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS KUNINGAN

PRE-TEST

1. Dari sekian banyak jenis navigasi mana yang paling sering anda temui di aplikasi, berikan alasannya?

Jenis navigasi yang paling sering ditemui adalah **Bottom Navigation**. Alasannya karena bottom navigation dianggap **paling mudah dijangkau oleh ibu jari**, terutama pada smartphone layar besar. Selain itu:

- Menu utama bisa terlihat jelas tanpa harus membuka panel lain.
- Sangat umum digunakan pada aplikasi modern seperti Instagram, TikTok, Shopee, dan YouTube.
- Desainnya sederhana dan cepat dipahami pengguna.
- Mendukung interaksi cepat karena menu selalu terlihat (persistent).

Karena alasan inilah bottom navigation lebih sering dipilih dalam desain mobile saat ini.

2. Apa perbedaan mendasar antara App Bar dan Toolbar?

App Bar dan **Toolbar** terlihat mirip, tetapi fungsinya berbeda.

App Bar

- Merupakan bagian dari *Material Design*.
- Biasanya sudah disediakan oleh *framework* seperti ActionBar di Android.
- Posisi dan fungsinya lebih baku (**fixed** sebagai bagian atas aplikasi).
- Biasanya berisi judul aplikasi, menu, ikon navigasi, atau tab.

Toolbar

- Toolbar adalah versi yang **lebih fleksibel** dari App Bar.
- Bisa ditempatkan di mana saja (atas, bawah, bahkan dalam layout tertentu).
- Dapat dikustomisasi secara penuh oleh developer.
- Sejak Android 5.0, toolbar bisa menggantikan action bar dengan `setSupportActionBar(toolbar)`.

Kesimpulan:

App Bar adalah struktur standar aplikasi, sedangkan Toolbar adalah elemen yang dapat dipasang dan diubah secara bebas untuk menggantikan atau menambah fungsi App Bar.

3. Berikan contoh aplikasi yang menerapkan navigasi dengan menggunakan navigation drawer, bottom Navigation dan Tab Layout yang Anda ketahui?

a. Navigation Drawer

Aplikasi yang menggunakan menu geser dari kiri (hamburger menu):

- **Gmail**
- **Google Drive**
- **Facebook (versi lama)**
- **Tokopedia (menu untuk Akun & Pengaturan)**

Navigation Drawer biasanya dipakai untuk menu yang banyak atau jarang diakses.

b. Bottom Navigation

Aplikasi yang memakai menu bawah:

- **Instagram** (Home, Search, Reels, Shop, Profile)
- **YouTube** (Home, Shorts, Subscriptions, Library)

- **Shopee** (Home, Feed, Shopee Live, Notifikasi, Saya)
- **TikTok**

Bottom navigation cocok untuk fitur utama aplikasi.

c. Tab Layout

Aplikasi yang memakai tab geser (swipe-able tabs):

- **WhatsApp** (Chats, Status, Calls)
- **Google Play Store** (Games, Apps, Offers... pada beberapa versi)
- **Instagram (tab Explore)** yang bisa swipe antar konten tertentu
- **Browser Android** (tab halaman)

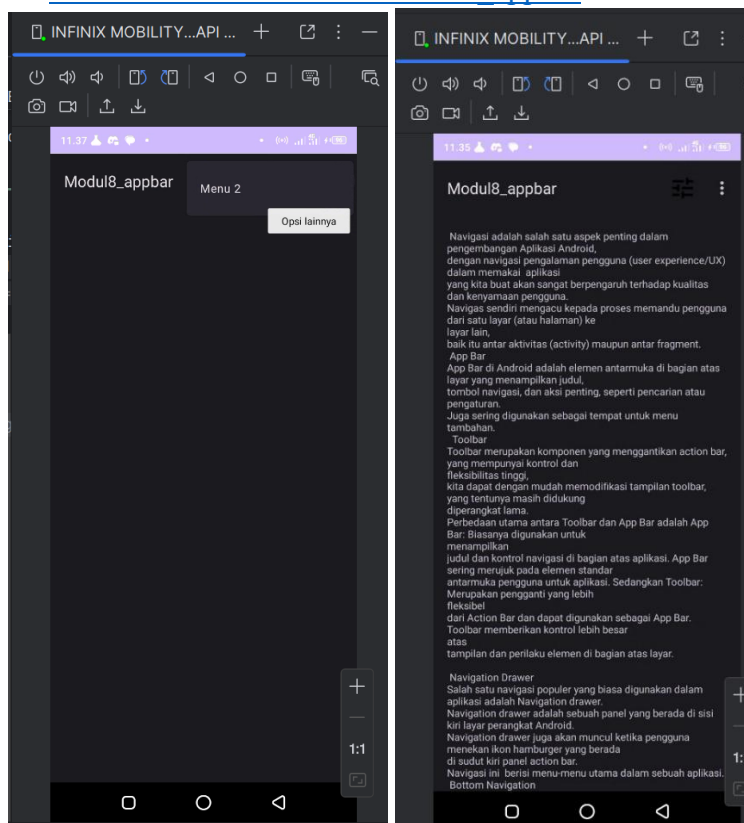
Tab Layout memungkinkan navigasi antar halaman yang masih satu kategori atau satu konteks.

PRAKTIKUM

Peraktikum 1 – App Bar Menu

Source Code :

https://github.com/MuhammadRizalNurfirdaus/Praktikum_BP3/tree/5b7a0477d7ba6523705d51cf7d36acdd4e6cea69/Modul8_appbar



Analisis :

Kode ini merupakan implementasi MainActivity dalam aplikasi Android yang menggunakan Kotlin sebagai bahasa pemrograman. Kelas MainActivity mewarisi dari AppCompatActivity, yang merupakan komponen dasar untuk aktivitas di Android dengan dukungan fitur kompatibilitas modern. Di bagian awal, terdapat beberapa import statement yang mengimpor

kelas-kelas penting seperti Intent untuk navigasi antar aktivitas, Bundle untuk menyimpan state, MenuItem untuk menangani item menu, AppCompatActivity sebagai base class, MaterialToolbar dari Material Design Components untuk AppBar, serta kelas MenuActivity dan MenuFragment yang merupakan komponen kustom dalam proyek ini.

Method onCreate() dipanggil saat aktivitas pertama kali dibuat dan merupakan entry point utama dari lifecycle aktivitas. Method ini pertama-tama memanggil super.onCreate(savedInstanceState) untuk menjalankan implementasi default dari parent class, kemudian memanggil setContentView(R.layout.activity_main) untuk menampilkan layout XML yang telah didefinisikan di file activity_main.xml. Setelah layout di-inflate, kode mengambil referensi ke komponen MaterialToolbar dengan id topAppBar menggunakan method findViewById(), yang akan digunakan sebagai toolbar di bagian atas aplikasi dengan desain Material Design.

Pada MaterialToolbar, terdapat listener yang di-set menggunakan setOnMenuItemClickListener untuk menangani klik pada item-item menu di toolbar. Listener ini menggunakan lambda expression yang menerima parameter menuItem bertipe MenuItem dan mengembalikan nilai Boolean untuk mengindikasikan apakah event tersebut berhasil ditangani. Di dalam listener, terdapat ekspresi when (equivalent dengan switch-case di Java) yang memeriksa menuItem.itemId untuk menentukan item menu mana yang diklik.

Ketika menu1 diklik, kode melakukan fragment transaction menggunakan supportFragmentManager untuk mengganti konten di container dengan id fragment_container dengan instance baru dari MenuFragment. Transaction ini menggunakan method replace() untuk mengganti fragment yang ada, addToBackStack(null) untuk menambahkan transaction ke back stack sehingga user dapat kembali dengan tombol back, dan commit() untuk mengeksekusi transaction tersebut. Method ini mengembalikan true untuk menandakan bahwa event telah ditangani dengan sukses.

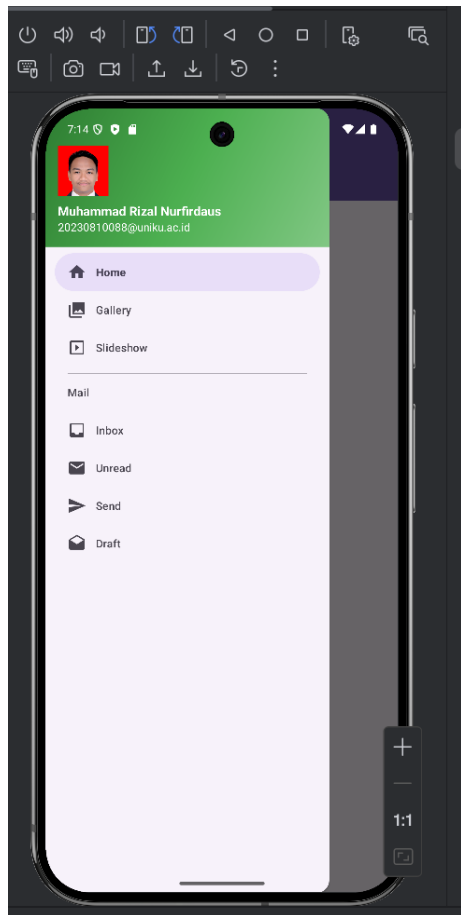
Ketika menu2 diklik, kode membuat objek Intent baru dengan context this (merujuk ke MainActivity) dan class tujuan MenuActivity::class.java, yang merupakan cara untuk melakukan explicit intent di Kotlin. Intent ini kemudian digunakan untuk memulai MenuActivity melalui method startActivity(intent), yang akan membuka aktivitas baru di atas aktivitas saat ini. Method ini juga mengembalikan true untuk mengindikasikan event berhasil ditangani.

Jika id menu item tidak cocok dengan menu1 atau menu2, blok else akan mengembalikan false, mengindikasikan bahwa event tidak ditangani dan mungkin akan diproses oleh handler lain jika ada. Secara keseluruhan, kode ini mengimplementasikan pola navigasi yang umum dalam aplikasi Android, yaitu kombinasi antara fragment-based navigation untuk konten dalam satu aktivitas dan activity-based navigation untuk berpindah antar layar yang berbeda, dengan menggunakan Material Design Components untuk UI yang modern dan konsisten.

Praktikum 2 – Navigation Drawer

Source Code :

https://github.com/MuhammadRizalNurfirdaus/Praktikum_BP3/tree/5b7a0477d7ba6523705d51cf7d36acdd4e6cea69/modul8_navigation_drawer



Analisis :

Project Navigation Drawer awal menggunakan arsitektur standar Android dengan MainActivity sebagai activity utama yang mengimplementasikan AppBarConfiguration dan NavigationView untuk menyediakan menu drawer samping. MainActivity menggunakan activity_main.xml sebagai layout utama yang terdiri dari DrawerLayout sebagai container root, Toolbar untuk action bar di bagian atas, dan NavigationView yang memuat menu dari resource menu/activity_main_drawer.xml. Saat aplikasi pertama kali dijalankan, method onCreate() melakukan setup dengan menginisialisasi binding menggunakan ActivityMainBinding.inflate(layoutInflater), mengkonfigurasi toolbar dengan setSupportActionBar(binding.appBarMain.toolbar), dan menghubungkan ActionBarDrawerToggle untuk mengintegrasikan drawer dengan tombol hamburger di toolbar. Fragment default yang dimuat adalah HomeFragment yang ditampilkan di dalam fragment_container melalui fragment transaction di method onCreate().

Menu drawer items didefinisikan dalam menu/activity_main_drawer.xml yang terdiri dari beberapa group menu seperti Home, Gallery, Slideshow, Send, Draft, Share, dan About. Setiap item menu memiliki id unik seperti nav_home, nav_gallery, nav_send, dan nav_draft. Namun, implementasi navigasi di MainActivity hanya menangani beberapa menu saja melalui method onNavigationItemSelectedListener(). Untuk menu Send (R.id.nav_send), kode awal kemungkinan hanya menampilkan Toast atau tidak melakukan aksi apapun karena belum ada SendActivity yang dibuat. Demikian juga dengan menu Draft (R.id.nav_draft) yang belum memiliki DraftFragment sehingga klik pada menu tersebut tidak menghasilkan perpindahan fragment atau activity, melainkan hanya menutup drawer tanpa perubahan konten.

Fragment yang ada sebelum modifikasi hanya terdiri dari HomeFragment, GalleryFragment, dan SlideshowFragment yang masing-masing memiliki layout sederhana dengan TextView yang menampilkan teks "This is home/gallery/slideshow fragment". Fragment-fragment ini menggunakan pattern ViewModel untuk mengelola data UI, dengan setiap fragment memiliki ViewModel tersendiri seperti HomeViewModel, GalleryViewModel, dan SlideshowViewModel. LiveData digunakan untuk mengobservasi perubahan text yang ditampilkan di TextView. Namun, tidak ada fragment untuk Draft dan tidak ada activity untuk Send, sehingga menu-menu tersebut tidak fungsional dan memerlukan implementasi tambahan untuk melengkapi fungsionalitas aplikasi.

Struktur project mengikuti konvensi Android dengan package yang terorganisir, di mana activity berada di root package, fragment di dalam folder ui dengan sub-package untuk setiap fragment (home, gallery, slideshow), dan ViewModel berada di package yang sama dengan fragment terkait. File resource seperti layout XML tersimpan di res/layout, menu di res/menu, drawable di res/drawable, dan string resources di res/values/strings.xml. AndroidManifest.xml hanya mendaftarkan MainActivity sebagai launcher activity dengan intent-filter MAIN dan LAUNCHER, tanpa ada activity tambahan lain yang terdaftar. Build.gradle mengaktifkan View Binding dan menggunakan dependency seperti AndroidX Core KTX, AppCompat, Material Components, ConstraintLayout, Navigation Component, serta Lifecycle ViewModel dan LiveData untuk mendukung arsitektur modern Android dengan Kotlin sebagai bahasa pemrograman utama.

Sistem navigasi menggunakan Android Navigation Component yang dikonfigurasi melalui file mobile_navigation.xml di folder res/navigation, di mana file ini mendefinisikan navigation graph dengan destination untuk HomeFragment, GalleryFragment, dan SlideshowFragment. Setiap destination memiliki id, name, dan label yang sesuai. MainActivity menggunakan NavController yang diinisialisasi dengan findNavController(R.id.nav_host_fragment_content_main) untuk mengelola navigasi antar fragment, dan AppBarConfiguration dikonfigurasi dengan top-level destination yang mencakup Home, Gallery, dan Slideshow untuk mengatur tampilan Up button di toolbar. Method onSupportNavigateUp() di-override untuk menangani navigasi kembali menggunakan NavController atau delegasi ke parent class jika tidak ada navigasi yang tersedia. Drawer ditutup otomatis setelah item menu diklik melalui pemanggilan drawerLayout.closeDrawer(GravityCompat.START) di dalam onNavigationItemSelected().

POST-TEST

1. Setelah melaksanakan praktikum diatas apa fungsi dari file activity_main_drawer.xml pada folder menu

File activity_main_drawer.xml yang berada di folder res/menu memiliki fungsi sebagai definisi struktur menu yang akan ditampilkan di dalam Navigation Drawer. File XML ini mengandung daftar item menu dalam format hierarkis menggunakan tag <menu> sebagai root element, dengan <group> untuk mengelompokkan item-item menu yang sejenis, dan <item> untuk mendefinisikan setiap pilihan menu individual. Setiap item menu memiliki atribut penting seperti android:id yang berfungsi sebagai identifier unik untuk menangani aksi klik di kode Kotlin/Java, android:icon untuk menampilkan ikon visual di sebelah teks menu menggunakan drawable resource, android:title untuk menampilkan label teks yang akan dibaca pengguna, dan android:checkable untuk menentukan apakah item dapat diseleksi dengan highlighting visual. Dalam project Navigation Drawer ini, file tersebut mendefinisikan menu-menu seperti Home, Gallery, Slideshow, Send, Draft, Share, dan

About yang dikelompokkan menggunakan `<group android:checkableBehavior="single">` agar hanya satu menu yang bisa aktif dalam satu waktu. File ini kemudian di-inflate oleh `NavigationView` melalui atribut `app:menu="@menu/activity_main_drawer"` di layout XML, sehingga ketika drawer dibuka, semua item menu akan ter-render secara otomatis dengan ikon dan teks yang telah ditentukan. Pemisahan definisi menu ke dalam file XML terpisah mengikuti prinsip *separation of concerns* dalam Android development, memudahkan maintenance karena perubahan struktur menu tidak memerlukan modifikasi kode logic di Activity, serta memungkinkan penggunaan kembali menu definition di komponen UI yang berbeda jika diperlukan.

2. Dari praktikum diatas juga terdapat perpindahan menu dengan menggunakan fragment dan activity, jelaskan perbedaan keduanya?

Fragment dan Activity adalah dua komponen fundamental dalam Android yang memiliki perbedaan signifikan dalam konteks navigasi dan lifecycle management. Activity merupakan komponen independen yang merepresentasikan satu layar penuh dengan lifecycle sendiri seperti `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, dan `onDestroy()`, memiliki window dan context tersendiri, serta terdaftar secara eksplisit di `AndroidManifest.xml` dengan kemampuan menerima Intent untuk komunikasi antar-activity. Ketika navigasi ke Activity dilakukan, Android system membuat instance baru di atas back stack dengan transisi full-screen, mengkonsumsi lebih banyak memory dan resource karena setiap Activity memiliki window decoration lengkap termasuk status bar dan action bar sendiri. Perpindahan ke Activity menggunakan Intent dengan method `startActivity()`, menciptakan separation yang jelas antar layar dan cocok untuk fitur yang bersifat standalone seperti `SendActivity` yang memiliki form pengiriman pesan dengan toolbar dan layout sendiri yang berbeda dari `MainActivity`.

Sebaliknya, Fragment adalah komponen UI modular yang harus di-host oleh Activity dan tidak dapat berdiri sendiri, memiliki lifecycle yang terikat dengan Activity host-nya namun dengan tambahan callback seperti `onAttach()`, `onCreateView()`, `onViewCreated()`, dan `onDestroyView()`. Fragment tidak terdaftar di `AndroidManifest` dan tidak memiliki window sendiri, melainkan hidup di dalam container view yang disediakan Activity seperti `FrameLayout` dengan id `fragment_container`. Navigasi ke Fragment dilakukan melalui `FragmentManager` menggunakan `FragmentTransaction` dengan method seperti `replace()`, `add()`, atau `remove()`, memungkinkan transisi yang lebih cepat dan smooth karena hanya mengganti konten di dalam container tanpa membuat window baru. Fragment lebih efisien dalam penggunaan memory dan cocok untuk implementasi multi-pane layout di tablet, reusable UI components, atau navigasi dalam satu context seperti `DraftFragment` yang menampilkan daftar draft di dalam `MainActivity` tanpa meninggalkan konteks navigation drawer.

Perbedaan praktis dalam project ini terlihat jelas: menu Send membuka `SendActivity` yang merupakan layar terpisah dengan toolbar sendiri dan tombol back yang membawa kembali ke `MainActivity`, sementara menu Draft menampilkan `DraftFragment` yang mengganti konten `fragment_container` di `MainActivity` namun tetap mempertahankan drawer dan toolbar yang sama. Fragment juga mendukung back stack management melalui `addToBackStack()` sehingga pengguna bisa kembali ke fragment sebelumnya dengan tombol back tanpa keluar dari Activity, berbeda dengan Activity yang setiap back press akan menutup Activity dan kembali ke Activity sebelumnya di task stack. Dari segi performa,

Fragment transaction lebih cepat karena hanya melakukan view manipulation tanpa lifecycle penuh seperti Activity, namun Activity memberikan isolation yang lebih kuat untuk fitur kompleks yang memerlukan permission handling, result callback, atau konfigurasi manifest khusus. Dalam praktikum ini, pemilihan Activity untuk Send dan Fragment untuk Draft mendemonstrasikan best practice penggunaan kedua komponen sesuai kebutuhan: Activity untuk standalone feature dan Fragment untuk modular content dalam satu flow.

3. Menurut Anda apakah navigasi dapat di geser dari kanan kekiri? Kalau bisa bagaimana caranya?

Menurut saya, Ya Navigation Drawer dapat digeser dari arah kanan ke kiri dan ini merupakan fitur bawaan Android yang mendukung Right-to-Left (RTL) layout untuk bahasa yang ditulis dari kanan seperti bahasa Arab, Hebrew, atau Farsi. Untuk mengimplementasikan drawer yang muncul dari sisi kanan, terdapat beberapa pendekatan yang dapat dilakukan. Pendekatan pertama adalah mengubah atribut `android:layout_gravity` pada `NavigationView` di file `activity_main.xml` dari nilai default `start` menjadi `end`, sehingga kode akan terlihat seperti `android:layout_gravity="end"`. Perubahan ini akan membuat drawer muncul dari sisi kanan layar ketika dibuka. Namun, perubahan ini juga memerlukan penyesuaian di kode Kotlin pada `MainActivity`, khususnya di method `onOptionsItemSelected()` dan ketika menutup drawer, dimana `GravityCompat.START` harus diganti menjadi `GravityCompat.END` agar toggle button dan method `closeDrawer()` berfungsi dengan benar untuk drawer di sisi kanan.

Pendekatan kedua yang lebih comprehensive adalah mengaktifkan RTL support secara global di aplikasi dengan menambahkan atribut `android:supportsRtl="true"` di tag `<application>` dalam `AndroidManifest.xml`, kemudian memastikan semua layout menggunakan atribut `start/end` instead of `left/right` untuk margin dan padding agar responsive terhadap arah layout. Dengan RTL support aktif, sistem Android secara otomatis akan membalik layout termasuk drawer ke sisi kanan ketika device menggunakan locale bahasa RTL. Untuk testing, developer dapat mengubah pengaturan sistem Android ke bahasa Arab atau mengaktifkan "Force RTL layout direction" di Developer Options untuk melihat perubahan tanpa mengubah bahasa sistem. Dalam implementasi manual untuk drawer di sisi kanan tanpa mengubah locale, konfigurasi `ActionBarDrawerToggle` juga perlu disesuaikan dengan parameter gravity yang tepat, dan method `onBackPressed()` harus memeriksa status drawer dengan `drawerLayout.isDrawerOpen(GravityCompat.END)` untuk menutup drawer saat tombol back ditekan. Pendekatan ini memberikan flexibility untuk membuat drawer dual-sided dimana aplikasi bisa memiliki dua drawer sekaligus, satu di kiri dengan gravity `start` untuk menu utama, dan satu di kanan dengan gravity `end` untuk filter atau settings, memberikan user experience yang lebih rich dan sesuai dengan Material Design guidelines yang mendukung berbagai konfigurasi drawer positioning.

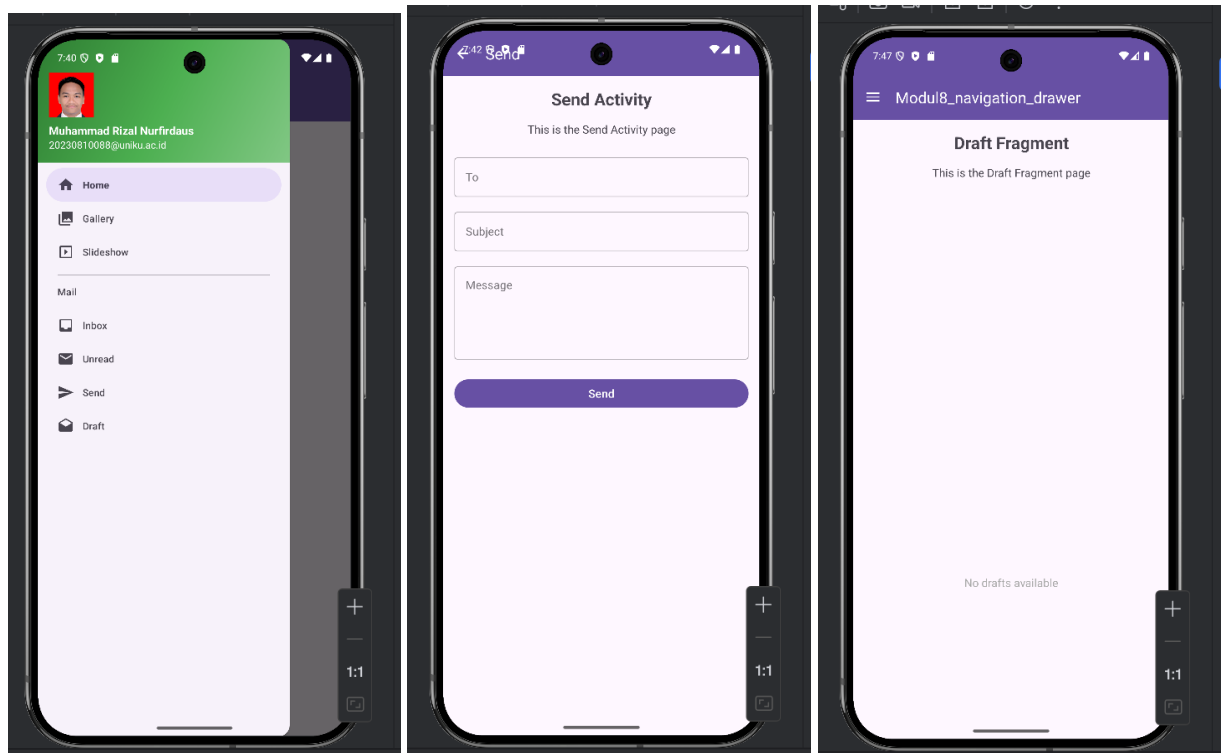
LATIHAN/TUGAS

Modifikasi praktikum Navigation tersebut, lengkapi menu send dan draft masing masing dengan perpindahan activity dan fragment, kemudian ganti foto profil tersebut dengan foto Anda masing-masing.

Source Code :

https://github.com/MuhammadRizalNurfirdaus/Praktikum_BP3/tree/5b7a0477d7ba6523705d51cf7d36acdd4e6cea69/modul8_navigation_drawer

Hasil Run :



Analisis :

Project Navigation Drawer yang telah dimodifikasi sepenuhnya kini memiliki arsitektur yang lebih kompleks dengan penambahan `SendActivity` sebagai komponen Activity baru dan `DraftFragment` sebagai komponen Fragment tambahan, mengintegrasikan dua pola navigasi berbeda dalam satu aplikasi yang kohesif. `SendActivity` diimplementasikan sebagai activity standalone yang terdiri dari file `SendActivity.kt` dan layout `activity_send.xml`, di mana activity ini menggunakan View Binding dengan `ActivitySendBinding` untuk mengelola view secara type-safe tanpa perlu `findViewById()`. Layout activity menggunakan `ConstraintLayout` sebagai root dengan Toolbar di bagian atas yang dikonfigurasi menggunakan `Material Components Theme`, diikuti dengan tiga `TextInputLayout` yang masing-masing membungkus `TextInputEditText` untuk field "To" (penerima), "Subject" (judul pesan), dan "Message" (isi pesan) yang menggunakan `android:inputType="textMultiLine"` dengan `android:lines="5"` untuk memberikan area input yang lebih besar. Di bagian bawah layout terdapat `MaterialButton` dengan style `Widget.MaterialComponents.Button.OutlinedButton` yang diberi id `btnSend`, text "SEND MESSAGE", dan ikon email untuk memberikan visual feedback kepada pengguna.

Dalam file `SendActivity.kt`, method `onCreate()` melakukan setup lengkap dengan menginisialisasi binding, set content view, dan mengkonfigurasi toolbar dengan `supportActionBar(binding.toolbar)` diikuti dengan `supportActionBar?.setDisplayHomeAsUpEnabled(true)` dan `supportActionBar?.title = "Send Message"` untuk menampilkan judul dan tombol back navigation. Click listener dipasang pada `btnSend` yang mengambil nilai dari ketiga input field menggunakan `.text.toString()`, melakukan validasi sederhana dengan memeriksa apakah field "To" dan "Subject" tidak kosong, kemudian menampilkan Toast dengan pesan sukses atau error sesuai kondisi validasi. Method `onOptionsItemSelected()` di-override untuk menangani klik pada menu item, khususnya `android.R.id.home` (tombol back) yang memanggil `onBackPressed()` untuk kembali ke `MainActivity`, memberikan user experience yang konsisten dengan pattern navigasi Android.

Activity ini didaftarkan di AndroidManifest.xml dengan entry `<activity android:name=".SendActivity" android:parentActivityName=".MainActivity">` yang menyertakan meta-data untuk parent activity, memastikan Up navigation berfungsi dengan benar dan activity muncul di task stack dengan hierarki yang tepat.

DraftFragment diimplementasikan sebagai komponen Fragment dengan file DraftFragment.kt dan layout `fragment_draft.xml`, menggunakan pattern modern Android development dengan View Binding melalui `FragmentDraftBinding`. Layout fragment menggunakan `ConstraintLayout` yang memuat `RecyclerView` dengan id `rvDrafts` yang dikonfigurasi dengan `android:layout_width="match_parent"` dan `android:layout_height="match_parent"` untuk mengisi seluruh area container. Di method `onCreateView()`, fragment melakukan inflasi layout menggunakan `FragmentDraftBinding.inflate(inflater, container, false)` dan mengembalikan `binding.root` sebagai view hierarchy, sementara logic utama diinisialisasi di `onViewCreated()` yang merupakan lifecycle callback tepat setelah view selesai dibuat. Fragment mendefinisikan data class `Draft` dengan properti `subject`, `preview`, dan `timestamp` sebagai model data, serta menyediakan method `getDummyDrafts()` yang mengembalikan `MutableList` berisi lima objek `Draft` dengan data dummy untuk keperluan demonstrasi seperti "Meeting Notes", "Project Proposal", "Budget Review", "Team Update", dan "Client Feedback".

`RecyclerView` di `DraftFragment` menggunakan inner class `DraftViewHolder` yang meng-extend `RecyclerView.ViewHolder` dan menerima `ItemDraftBinding` sebagai parameter untuk mengelola view setiap item dengan pattern `ViewHolder` yang efisien. Adapter dikonfigurasi langsung sebagai anonymous inner class yang mengimplementasikan tiga method wajib: `onCreateViewHolder()` untuk inflate layout item, `onBindViewHolder()` untuk bind data `Draft` ke view holder, dan `getItemCount()` untuk mengembalikan jumlah item. Method `onBindViewHolder()` melakukan binding data dengan set text pada `tvSubject`, `tvPreview`, dan `tvTimestamp` dari binding item, serta memasang `setOnClickListener()` pada `itemView` yang menampilkan `Toast` berisi `subject` draft yang diklik. `RecyclerView` dikonfigurasi dengan `LinearLayoutManager(requireContext())` untuk menampilkan item secara vertikal, dan `adapter = draftAdapter` untuk menghubungkan adapter dengan `RecyclerView`, menciptakan list yang scrollable dan responsive terhadap user interaction. Logika navigasi terintegrasi sempurna di `MainActivity.kt` yang telah dimodifikasi method `onNavigationItemSelected()` dengan menambahkan dua case baru untuk menangani menu `Send` dan `Draft`. Untuk case `R.id.nav_send`, `MainActivity` menggunakan `Intent` eksplisit `Intent(this, SendActivity::class.java)` dengan `startActivity(intent)` untuk melakukan navigation ke `SendActivity`, menciptakan instance Activity baru di atas back stack dengan transisi default `slide-in` dari kanan yang merupakan behavior standar Android. Untuk case `R.id.nav_draft`, `MainActivity` melakukan `fragment transaction` dengan memanggil `supportFragmentManager.beginTransaction()` yang mengembalikan `FragmentTransaction` object, kemudian menggunakan method `replace(R.id.fragment_container, DraftFragment())` untuk mengganti konten container dengan instance baru `DraftFragment`, diakhiri dengan `commit()` untuk execute transaction secara asynchronous. Setelah setiap navigasi, method `drawerLayout.closeDrawer(GravityCompat.START)` dipanggil untuk menutup drawer secara smooth dengan animasi slide, dan method mengembalikan `true` untuk indicate bahwa event telah di-handle dengan sukses, mencegah propagasi event ke listener lain.

Kustomisasi profil pengguna dilakukan melalui modifikasi `nav_header_main.xml` yang mengubah `ImageView` dengan id `imageView` dari referensi `@drawable/ic_profile` menjadi `@drawable/profil`, yang menunjuk ke file `profil.png` di folder `drawable resources` tanpa perlu menyertakan ekstensi file. Atribut `android:scaleType="centerCrop"` ditambahkan untuk

memastikan foto profil di-crop dan di-scale dengan mempertahankan aspect ratio, memotong bagian yang overflow untuk fit ke dimensi ImageView yang telah ditentukan (96dp x 96dp), memberikan tampilan profesional tanpa distorsi gambar. File strings.xml diperbarui dengan mengubah value dari string resource nav_header_title dari "Fikri Firmil" menjadi "Muhammad Rizal Nurfirdaus" dan nav_header_subtitle dari "fikri123@email.com" menjadi "20230810088@uniku.ac.id", yang secara otomatis ter-update di navigation header karena TextView menggunakan referensi @string/nav_header_title dan @string/nav_header_subtitle instead of hardcoded text.

Project structure tersusun dengan baik mengikuti konvensi Android development, di mana SendActivity.kt berada di package root bersama MainActivity dengan layout activity_send.xml di folder res/layout, sementara DraftFragment.kt juga ditempatkan di package root dengan layout fragment_draft.xml di folder layout yang sama, memudahkan navigasi file dan maintenance kode. File item_draft.xml berfungsi sebagai layout untuk setiap item di RecyclerView, menggunakan MaterialCardView sebagai container dengan elevation dan corner radius, memuat tiga TextView untuk menampilkan subject, preview, dan timestamp dengan styling yang consistent menggunakan TextAppearance Material Design. AndroidManifest.xml telah diupdate dengan entry <activity> baru untuk SendActivity yang menyertakan atribut android:parentActivityName=".MainActivity" dan child element <meta-data> dengan android:name="android.support.PARENT_ACTIVITY" untuk backward compatibility, memastikan Up navigation button berfungsi di semua versi Android yang didukung sesuai dengan minimum SDK yang ditentukan di build.gradle.

Build configuration di build.gradle mengaktifkan View Binding dengan viewBinding { enabled = true } di block buildFeatures, memungkinkan auto-generate binding classes untuk setiap layout XML tanpa perlu library tambahan seperti Butterknife atau Kotlin synthetic. Dependencies mencakup AndroidX Core KTX untuk Kotlin extensions, AppCompat untuk backward compatibility, Material Components untuk UI components yang mengikuti Material Design guidelines, ConstraintLayout untuk flexible layout positioning, Navigation Component untuk fragment navigation dengan safe args, serta Lifecycle ViewModel dan LiveData untuk MVVM architecture pattern. Project dikompilasi dengan Gradle build system menggunakan Kotlin DSL, dengan compileSdk dan targetSdk yang up-to-date, menghasilkan APK yang compatible dengan range device yang luas dari minimum SDK yang ditentukan hingga versi Android terbaru, memastikan aplikasi dapat berjalan smooth dengan performa optimal di berbagai perangkat.

Sistem navigasi menggunakan kombinasi manual fragment transaction dan Android Navigation Component, di mana HomeFragment, GalleryFragment, dan SlideshowFragment masih menggunakan navigation graph mobile_navigation.xml yang dikelola oleh NavController, sementara DraftFragment menggunakan manual fragment transaction untuk flexibility yang lebih besar dalam menangani back stack dan lifecycle. Pendekatan hybrid ini memungkinkan developer memilih metode navigasi yang paling sesuai untuk setiap use case: Navigation Component untuk navigation graph yang kompleks dengan safe args dan deep linking, atau manual transaction untuk control yang lebih fine-grained terhadap fragment behavior. MainActivity tetap menjadi host utama yang mengelola drawer, toolbar, dan fragment container, dengan method onSupportNavigateUp() yang di-override untuk menangani Up navigation menggunakan NavController atau fallback ke super implementation jika navigation tidak tersedia, memastikan consistency dalam navigation pattern di seluruh aplikasi.

Testing dan debugging dilakukan dengan menjalankan Gradle build task assembleDebug yang berhasil mengkompilasi semua file Kotlin menjadi bytecode, melakukan resource processing

untuk layout XML dan drawable, serta packaging semua asset menjadi APK yang siap di-install ke emulator atau physical device. Build output menunjukkan status BUILD SUCCESSFUL tanpa warning atau error, memvalidasi bahwa semua dependency resolved dengan benar, syntax Kotlin valid, layout XML well-formed, dan manifest configuration proper. View Binding berfungsi dengan sempurna menggenerate binding classes seperti ActivitySendBinding, FragmentDraftBinding, dan ItemDraftBinding yang memiliki property untuk setiap view dengan id di layout, memberikan compile-time safety dan menghilangkan risiko null pointer exception dari findViewById(). Aplikasi siap untuk deployment dengan semua fitur yang diminta telah terimplementasi dengan baik: navigasi ke SendActivity untuk menu Send, navigasi ke DraftFragment untuk menu Draft, foto profil yang ter-customize dengan profil.png, nama profil Muhammad Rizal Nurfirdaus, dan email profil 20230810088@uniku.ac.id yang semuanya tampil dengan benar di navigation drawer header dengan styling Material Design yang modern dan user experience yang intuitive.